

PERTEMUAN 5

ORGANISASI BERKAS LANGSUNG

A. Tujuan Pembelajaran

1. Memahami konsep dasar organisasi berkas langsung
2. Memahami macam – macam metode hashing
3. Mengerti tentang kunci sebagai alamat rekaman yang unik

B. Uraian Materi

Metode pencarian interpolasi atau biner masih tidak dianggap memenuhi kebutuhan manusia terhadap pemberi informasi yang baik. Pemakaian teknologi oleh manusia pada biasanya dan lebih tepatnya teknologi informasi dirasa bahwa waktu yang diperlukan untuk mendapatkan sebuah informasi masih sangat lambat.

Peningkatan teknik pencarian kembali data yang tersimpan harus seimbang dengan peningkatan alat record data untuk kemampuan atau menampung jumlah data yang sangat besar.

1. Kunci Sebagai Alamat Rekaman Yang Unik

Amat diharapkan supaya proses langsung menuju ke alamat tempat record menggunakan kunci tertentu disimpan. Untuk memperoleh record yang digabungkan dengan suatu kunci primer.

Hal tersebut dimungkinkan terjadi bila salah satu kunci record juga ialah suatu alamat lokasi rekaman. Untuk aplikasi record yang di dalamnya terdapat NIP atau nomor induk siswa, ada 10 digit (kunci rekaman yang merupakan pengkumpulan 6 digit tahun angkatan + 2 digit kode jurusan + 2 digit nomor urut), untuk memperoleh waktu pencarian akan lebih baik, yaitu 1 probe untuk setiap rekaman yang dicari. maka diperlukan lokasi sebanyak 99.999.999.

Akan tetapi, teknik tersebut mempunyai kekurangan karena di butuhkan ruang yang sangat besar untuk dapat menampung semua record. Untuk menampung data dari NIP siswa di perlukan ruang yang besar. Meskipun jumlah dari siswa mungkin hanya 600, pasti ada kurang lebih nomor yang tidak terisi karena ada beberapa alasan, contohnya sudah lulus, mengundurkan diri, cuti, dan lain-lain. dengan demikian tidak semua ruang digunakan.

Konversi Kunci Rekaman Menjadi Satu Alamat Yang Unik

Contoh yang sering digunakan untuk menggambarkan konversi catatan kunci adalah sistem pemesanan tiket. Jika nomor penerbangan maskapai adalah dari 1 hingga 1000, dan Anda ingin memantau reservasi penerbangan tahunan dengan 1 hingga 366 hari (1 tahun), Anda dapat menautkan nomor penerbangan dan tanggal kedatangan untuk mendapatkan catatan lokasi yang berisi data reservasi penerbangan untuk tanggal tertentu.

$$\text{LOKASI} = \text{NOMOR PENERBANGAN} + \text{HARI-KE DALAM TAHUN INI}$$

Gambar 5. 1 Nomor Penerbangan

Tanda + merupakan hubungan, maka untuk menyediakan semua kemungkinan penerbangan dibutuhkan ruang alamat sebesar 8888666 unit. Hasilnya tersebut dapat diperkecil sampai 30% bila memakai kombinasi:

$$\text{LOKASI} = \text{HARI-KE DALAM TAHUN INI} + \text{NOMOR PENERBANGAN}$$

Gambar 5. 2 Lokasi Hari

Karena kecil kemungkinan dalam satu maskapai terdapat lebih dari 150 penerbangan, maka ruang alamat maksimum yang diperlukan adalah 25588.

Menentukan Alamat Dengan Konversi Kunci

Dibutuhkan satu fungsi untuk meletakkan cakupan nilai kunci yang lebih besar ke dalam cakupan yang lebih kecil nilai dari alamat. Fungsi yang disebut tersebut yaitu fungsi hash.

Hash atau kunci memungkinkan alamat output hasil proses hashing tidak lagi alamat yang unik, tetapi kemungkinan alamat bagi kunci yang di hash.

Alamat untuk melokasikan alamat yang didapat dari fungsi hash ialah home-address bagi record tersebut. Tidak terbatas akan bentuk fungsi yang akan melokasikan kunci ke cakupan alamat, tetapi agar supaya fungsi tersebut membentuk kemungkinan alamat yang:

1. Dapat untuk mengurangi terbentuknya kelompok. pengkelompokan terjadi bila hasil hashing 2 kunci record yang berbeda menuju ke alamat yang sama. Dan juga dapat untuk membagikan kunci secara merata ke dalam cakupan alamat.
2. Dapat dilakukan dengan maksimal. Hal tersebut bertujuan agar waktu pembacaan dapat ditekan sebaik mungkin.

Maka fungsi hashing terdapat 2 aspek, ialah:

1. Fungsi hash itu sendiri
2. Metode untuk menyatakan kolisi

Resolusi kolusi dibutuhkan agar rekaman yang dirubah ke suatu lokasi yang memiliki kapasitas lebih besar. Di ketahui tulisan di atas menerangkan bahwa satu lokasi memiliki kapasitas satu rekaman.

2. Metode Hashing

Sistem Berkas langsung memiliki kelebihan yang maksimal, akan tetapi juga memiliki kekurangan.

Untuk menghilangkan kekurangannya itu maka digunakanlah metode lain yang di sebut metode hashing.

Dipergunakannya metode Hashing intinya ialah agar menurangi banyaknya ruang alamat yang dipakai dan untuk mengubah kunci yang mempunyai cangkupan yang besar ke nilai alamat yang mempunyai cangkupan yang kecil.

Skema Hashing

Ide utama di balik hashing adalah memasukkan setiap kunci yang dimuat ke dalam tabel ke dalam pohonnya sendiri dengan cara yang cerdas setiap pohon menerima sekitar jumlah kunci yang sama terlepas dari nilai-nilai mereka dan tanpa pengetahuan tentang distribusi aktual nilai-nilai ini.

Jika itu mungkin, kita akan memiliki kunci N / H di setiap pohon. Misalkan kita memiliki 2^{20} (sekitar 1 juta) kunci untuk memuat. Jika kita dimuat mereka dalam satu pohon (HASHEXP:0, $H = 1$), menggunakan pencarian pohon biner untuk menemukan atau menolak kunci pencarian akan memerlukan 21 perbandingan antara kunci pencarian dan beberapa kunci di pohon. Namun, jika kunci seragam dimuat ke pohon $H = 1024$ (HASHEXP:10), masing-masing akan berisi hanya sekitar 1024 kunci. Diberi kunci pencarian, kita akan tahu jumlah ember di mana ia berada. Mencari di antara 1024 kunci di pohon itu melalui pencarian biner kemudian akan memerlukan hanya 11 perbandingan kunci untuk menemukan atau menolak kunci pencarian; yaitu, kecepatan pencarian akan praktis dua kali lipat.

Fungsi Hashing Strategi membagi dan menaklukkan ini tidak sulit untuk dibayangkan. Tapi bagaimana kita memastikan bahwa setiap ember menerima kira-kira jumlah kunci input yang sama jika kita tidak tahu apa-apa tentang nilai-nilai mereka priori? Jika kunci input kami adalah, katakanlah, angka alami dari $KEY = 1$ ke $KEY = 1024$ dan kami memiliki 8 pohon untuk diisi, kami akan cukup bagi kunci dalam 8 rentang yang sama, dengan tepat 4 kunci per pohon. Untungnya, ada transformasi tertentu yang disebut fungsi hash. Fungsi hash yang baik memiliki tiga sifat dasar, yaitu :

- 1) Hal ini dapat menerima kunci sewenang-wenang N dengan nilai-nilai sewenang-wenang sebagai argumen dan mengembalikan nomor alami TN (nomor pohon) dari 1 hingga H : $TN = \text{hash_function}(KUNCI)$.
- 2) Setiap nomor TN dari 1 hingga H akan diberikan ke sekitar tombol N/H , dengan sangat sedikit atau tidak ada variasi antara nomor TN yang berbeda.
- 3) Untuk setiap nilai kunci yang diberikan, dapat mengembalikan satu dan hanya satu nilai TN . Dengan kata lain, akan ada tidak ada situasi ketika kunci yang sama ditetapkan ke dua pohon yang berbeda.

- 4) Hal ini cukup cepat untuk menghitung. Memang, jika sangat lambat untuk menghitung dan karenanya mengambil waktu yang sangat lama untuk menemukan pohon mana kunci milik, itu akan mengalahkan tujuan pencarian cepat dan efisien di tempat pertama.

Untuk melihat bagaimana prestasi seperti itu dapat ditarik, mari kita membentuk kunci komposit dari sepasang variabel (Team_SK, Player_ID) dari data set Bizarro.Player_candidates. Sekarang mari kita pasang semua 10.000 nilai yang berbeda ke dalam koktail dari fungsi bersarang yang digunakan sebagai fungsi hash di bawah ini untuk mendistribusikan nilai TN yang dihasilkan ke dalam angka dari 1 sampai 8. Key-indexed array Freq di bawah ini digunakan untuk mendapatkan frekuensi pada jumlah TN nilai ditempatkan ke dalam setiap ember.

Alasan kunci didistribusikan begitu merata adalah bahwa fungsi MD5 yang disediakan oleh SAS adalah hash fungsi itu sendiri. Di atas, mengkonsumsi kunci komposit (Team_SK, Player_ID) diubah menjadi karakter string dengan fungsi CATS dan menghasilkan string karakter 16 byte (yang disebut tanda tangan). PERINGKAT fungsi mengembalikan posisi byte pertama tanda tangan dalam urutan pemeriksaan. Akhirnya, MOD menggunakan pembagi 8 untuk mendistribusikan nomor posisi (mulai dari 0 hingga 255) antara nilai tn dari 1 sampai 8. Meskipun ada sekitar 4 persen variabilitas antara bucket yang paling banyak dan paling sedikit diisi, untuk semua maksud praktis dan tujuan menggunakan pencarian pohon biner dalam salah satu ember ini akan sama cepatnya. Berbeda dengan biner-mencari semua 10.000 kunci, itu akan menghemat sekitar 3 perbandingan kunci per pencarian biner sepenuhnya Ember.

Karena membandingkan kunci biasanya merupakan bagian yang paling komputasi pajak dari algoritma pencarian, mendistribusikan kunci di antara pohon-pohon mungkin membenarkan overhead komputasi TN dengan menggunakan hash fungsi di atas. Ekspresi yang diberikan di atas hanyalah salah satu contoh dari fungsi hash yang layak digunakan hanya untuk menggambarkan ide. Ia bekerja dengan baik karena MD5 itu sendiri adalah fungsi hash.

Meskipun fungsi hash internal bekerja untuk hash objek di balik layar berbeda, secara konseptual melayani tujuan yang sama untuk mendistribusikan kunci secara merata di seluruh jumlah pohon AVL yang dialokasikan.

Fungsi-fungsi Hash yang Sering Digunakan Dalam Hashing

Hashing dengan Kunci Modulus N

Fungsi hash yang paling sering digunakan adalah fungsi modulus N.

$$f(\text{kunci}) = \text{kunci} \bmod N$$

Gambar 5. 3 Fungsi Modulus N

Simbol N adalah sebuah ukuran suatu tabel. sisa pembagian kunci N adalah Hasil fungsi modulus. Salah satu contohnya, untuk $N=12$ maka: $30 \bmod N = 6$ $40 \bmod N = 4$ Keuntungan fungsi ini hanya menghasilkan nilai dalam rentang ruang alamat (0) sampai dengan (N-1)

Hashing dengan Kunci Modulus P

Fungsi selanjutnya ialah fungsi hashing modulus P adalah fungsi variasi dari fungsi modulus N, rumusnya yaitu :

$$f(\text{kunci}) = \text{kunci} \bmod P$$

Gambar 5. 4 Fungsi Modulus P

P sebagai bilangan prima terkecil yang lebih besar atau sama dengan N. dan N adalah ukuran tabel. P ini kemudian menjadi ukuran tabel baru yang menggantikan N. Contoh: untuk $N=12$ maka $P=13$ $30 \bmod P = 4$ $40 \bmod P = 1$

Hashing dengan Pemotongan

Fungsi lain dari metode hashing ialah pemotongan. Sebagai contoh yaitu Mahasiswa di suatu Universitas. Para Mahasiswa mempunyai NIM (Nomor Induk Mahasiswa) mempunyai 10 digit, terdiri dari 6 digit tahun angkatan dan 4 digit nomor urutnya.

Jika ingin dijadikan 8 digit saja, maka dapat diberlakukan pemotongan jumlah dari digit. Pemotongan dapat dilakukan pada bagian mana saja, tentunya dengan konsekuensi masing-masing.

Pada NIM, jika dilakukan pemotongan pada bagian belakang NIM, akan didapat 6 digit yang sama (tahun angkata) sehingga kemudian kolisnya akan lebih besar jika pemotongannya berada di bagian depan NIM.

Hashing dengan Lipatan

Hashing dengan lipatan ialah fungsi hashing yang dapat ditentukan dengan melihat kondisi digit bagian awal dan digit yang di hasilkan. Contohnya 9 digit NIM akan di perkecil menjadi 3 digit, maka digit bagian awal dibagi dengan 3, lalu dilipat pada batas antar bagian.

Contoh :

5	8	3	9	7	6	1	2	4
---	---	---	---	---	---	---	---	---

Gambar 5. 5 Hashing dengan Lipatan

Garis diatas adalah batasan yang dimana akan dilakukannya lipatan.

Jumlah dari susunan tersebut ialah::

385

976

421

.....+

Jika penjumlahan dilakukan dengan mengabaikan carry, hasil yang di dapat yaitu 672

Lalu jika tidak mengabaikan carry maka hasil yang didapat yaitu 782.

Hashing dengan Penggeseran

Hashing penggeseran mempunyai proses yang hampir sama dengan fungsi hashing lipatan, hanya saja perbedaannya yaitu sesudah ditentukan batasannya. Digit asli dipotong kemudian digeser dan di jumlahkan.

583

976

124

.....+

Yang didapat yaitu 573 (tanpa ada carry). Lalu jika seluruh hasil penjumlahan tersebut dijumlahkan dengan menggunakan carry dan hanya 3 digit di bagian akhir saja yang dipakai, maka hasil yang didapat 683.

Hashing dengan Pengkuadratan

Fungsi hashing dengan pengkuadratan yaitu fungsi hashing dilakukan dengan menguadratkan kunci, untuk mendapatkan alamat yang diperbolehkan dari hasil pengkuadratan ini kemudian dapat dilakukan penggabungan dengan pemotongan dan lipatan. Contohnya kunci 782 akan menghasilkan alamat 117. $F(782) = 117$.

Hashing dengan Konversi Radix

Dalam konversi radix, kunci dianggap dalam base selain 10 yang kemudian dikonversi ke dalam basis 10, misalkan kunci 5 6 7 8 didalam base 13 akan di

dapat 12098, diperoleh dari: 5 6 7 8, Posisi: 3 2 1 0 Hasil tersebut masih dapat digabungkan dengan fungsi hash lain seperti fungsi pemotongan atau lipatan untuk mendapatkan digit alamat yang diinginkan.

Struktur file akses langsung dibuat dengan tujuan untuk kebutuhan akses data secara langsung (Direct). Direct Access kadang kadang disebut dengan Random Access untuk menekankan bahwa dalam faktanya permintaan record itu sendiri tidak dalam urutan khusus.

Dalam struktur file akses langsung tiap record disimpan pada suatu lokasi yang dihitung dari nilai kunci utamanya. Semua program dengan penunjuk file harus mengetahui "Algoritma transformasi" yang nantinya digunakan untuk menghitung alamat awal penyimpanan dan semut penunjuk alamat untuk record.

Algoritma Transformasi menempatkan tiap kunci utama dan mengkonversinya menjadi alamat record relatif Relative Record Address (RRA). RRA adalah nomor antara 1-M, dimana M adalah nomor lokasi penyimpanan dalam file.

File akses langsung biasanya memerlukan ruang penyimpanan lebih daripada nomor data record itu sendiri karena:

- 1) Banyak sekelompok kunci utama bukan merupakan nomor sekuensial dari 1-N, dimana N = Nomor data record
- 2) Tidak ada Algoritma Transformasi yang bisa membuat mereka menjadi suatu himpunan rangkaian.

Hal ini menunjukkan bahwa file akses langsung memerlukan lebih daerah penyimpanan daripada file serial atau sequential untuk menyimpan data yang sama.

Masalah lain dari proses Algoritma Transformasi adalah bisa menghasilkannya RRA yang sama dari 2 nilai kunci utama. Record record dengan satu RRA yang sama disebut Synonyms, dan penyimpanannya memerlukan daerah Overflow, daerah yang kosong (free) untuk menyimpan record-record yang baru dan/atau yang synonyms.

Untuk mengakses file langsung, pemakai program harus memberikan nilai kunci utama untuk record yang diinginkan bagi algoritma transformasi. Dengan catatan suatu file akses langsung juga diakses melalui suatu rutin serial-access. Rutin ini harus menguji untuk record-record yang kosong dimana record dengan data yang tidak valid.

File-file akses langsung berguna hanya jika ketika salah satu record pada saat digunakan untuk pemrosesan dan ketika nilai kunci diketahui. Sistem Pengolahan secara On-Line seperti Sistem Pelayanan Rekening Bank, biasanya memakai tipe struktur ini. Untuk menuliskan laporan dari file akses langsung harus diproses dengan cara yang sama seperti file Serial.

3. Metode Akses pada File Akses Langsung (Direct Access File)

Alternatif penelusuran secara sekuensial suatu file untuk satu record pada mekanisme perolehan kembali record dikenal sebagai akses secara langsung.

Kita bisa mengakses record secara langsung satu record pada saat pencarian secara langsung di awal record dan membacanya. Dimana bila penelusuran secara sekuensial memerlukan operasi sebanyak $O(n)$, sedangkan untuk akses secara langsung hanya memerlukan operasi sebesar $O(1)$. Tidak menjadi soal berapa besar filenya, kita masih bisa mendapatkan record yang diinginkan dengan pencarian tunggal.

Permasalahan utama akses langsung diketahui di permulaan kebutuhan record adalah kadang-kadang informasi tentang lokasi record berada di file indeks yang terpisah, tetapi sesaat kita meng. asumsikan bahwa kita tidak memiliki suatu indeks. Dengan meng asumsikan bahwa kita mengetahui nomor relatif record (RRN) dari record yang kita inginkan.

RRN merupakan konsep penting yang timbul dari pandangan file sebagai sekumpulan record daripada se. bagai sekumpulan byte. Jika suatu file merupakan serangkaian record, RRN dari record memberikan posisi relatifnya ke awal file. Record pertama memiliki RRN 0, berikutnya RRN 1, dan seterusnya.

Di nama dan alamat file, kita bisa menghubungkan satu record dengan ke RRN-nya dengan menandai nomor keanggotaan yang di relasikan ke urutan dimana kita memasukkan record dalam file. Seseorang dengan record pertama memiliki nomor keanggotaan nomor 1001, nomor kedua 1002, dan seterusnya. Dengan memberikan nomor anggota, kita bisa mengurangi dengan 1001 untuk mendapat kan RRN suatu record.

RRN memberitahukan kepada kita posisi relatif record. Untuk mendukung akses langsung, kita harus bekerja dengan pemakaian record dengan panjang tetap. Jika semua record panjangnya sama, maka bisa menggunakan RRN record untuk menghitung byte offset di awal record relatif ke awal file. Contoh, jika menginginkan record dengan RRN 546 dan file yang kita memiliki ukuran panjang record tetap 128 byte per recordnya, maka byte offset bisa dihitung :

$$\text{Byte Offset} = 546 \times 128 = 69\,888$$

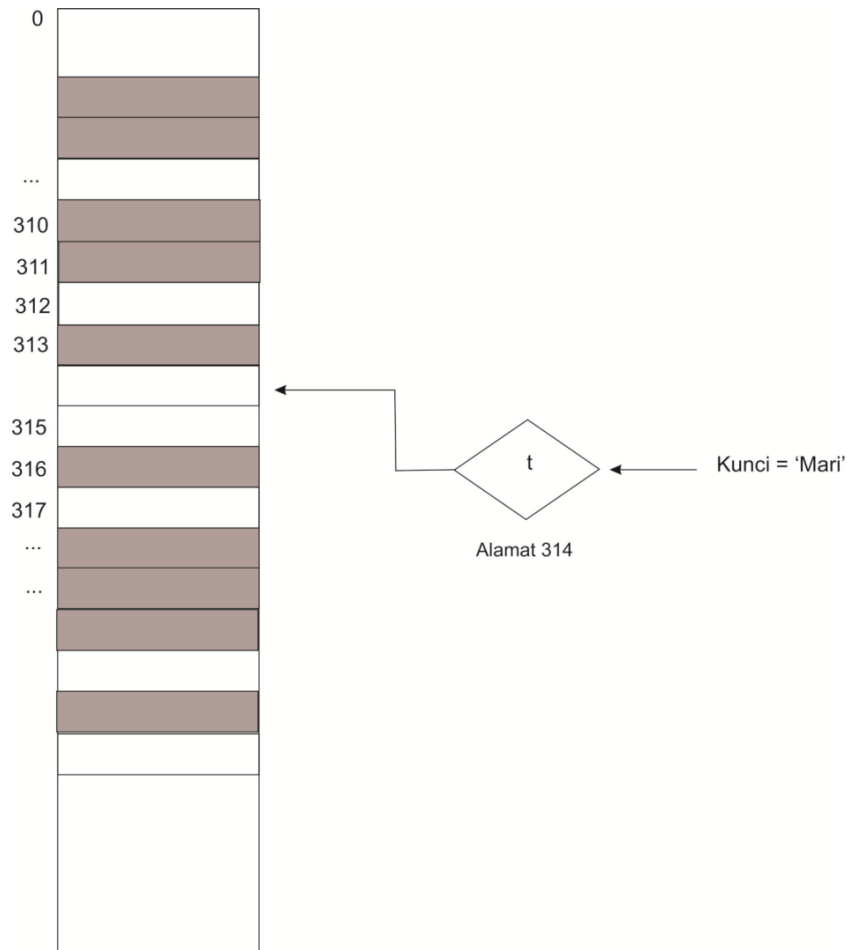
Bentuk umum record file dengan panjang tetap, ukuran record r , byte offset satu record dengan RRN n adalah :

$$\text{Byte offset} = n \times r$$

Metode file dengan akses langsung mentransformasikan kunci dengan menggunakan algoritma perhitungan sebelum digunakan sebagai suatu Address. Metode ini cepat selama bisa menghindari operasi file lanjutan, tetapi kekuatan dari metode ini adalah data di alokasikan sesuai dengan satu kunci atribut tunggal.

Bagaimanapun juga record dalam file langsung tidak di link ke record sebelumnya ataupun ke sesudahnya.

Direct File menggunakan ruang ekstra di file utama ($m > n$), untuk menjaga memberikan beberapa record ruang kosong di urutan ketika akan menyisipkan record baru ke file.



Gambar 5. 6 Metode Akses pada File Akses Langsung

4. Transformasi Kunci ke Address

Solusi permasalahan dengan menggunakan Key sebagai Alamat file yang meliputi pemakaian prosedur perhitungan yang menterjemahkan Key sebagai Alamat file yang meliputi pemakaian prosedur perhitungan yang menterjemahkan nilai Key-Attribute ke dalam "Relative Address" dalam ruang file.

Setiap calon Relative Address dikenali sebagai 1 slot dimana Batu record bisa ditempatkan.

Prosedur yang berbeda digunakan untuk menghitung alamat relatif dengan mengenali kunci seseorang dalam file karyawan dan dari kunci yang sama dalam file Sales-Personel. Prosedur ini tergantung pada tipe Kunci. Perhitungan ini disebut Transformasi Key- to-Address

Dua kategori perhitungan ini bisa berbeda. Ketetapan prosedur yang menterjemahkan identifikasi field ke bentuk alamat unik dan teknik pengacakan (Randomizing) yang menterjemahkan Kunci ke Alamat.

Prosedur Deterministik

Didapat dari himpunan semua nilai kunci dan menghitungnya menjadi sekumpulan alamat relatif yang khusus.

Transformasi Randomizing

Menterjemahkan Nilai-Kini menjadi Alamat Relatif menggunakan prosedur "Algoritma Randomizing"

Contoh : Transformasi Key-To-Address

Satu transformasi randomizing untuk satu file personel menggunakan nomor sosial_sekurity sebagai Key. Asumsi nilai dari digit orde rendah nomor ini didistribusikan dengan rata dan karenanya memungkinkan untuk memperoleh 3 digit nomor unik untuk setiap karyawan. Jika salah satu menginginkan ruang yang mungkin untuk 500 karyawan, nilai Kunci bisa dibagi dengan 500, menghasilkan sisa dengan nilai antara 0 sd 499.

Alamat yang sama bisa terjadi :

Ali	Atau	322-45-6178	memberikan address	178
Joe		123-45-6284		284
Marry		036-23-0373		373
Pete		901-23-4784		284

Gambar 5. 7 Transformasi Key-to-Address

Disini Joe dan Pete keduanya ditandai untuk satu record dengan nomor sama = 284 Record untuk Joe dan Pete akan bertubrukan jika keduanya di tempatkan secara langsung dalam satu file.

5. Hashing

Penentuan alamat suatu record pada file data dengan melakukan perhitungan pada file data dengan melakukan perhitungan pada Key-Value dari record tersebut. Yang ditulis sebagai :



Gambar 5. 8 Perhitungan Fungsi Transformasi

Dalam transformasi Key Value ke Address record, bisa timbul kemungkinan bahwa 2 record dengan Key-value yang berbeda di peta kan pada satu alamat record yang sama. Hal ini disebut dengan collision (Kolisi), yaitu :

$$H(K1)=H(K2) \text{ dimana } K1 \neq K2$$

Untuk mengatasi Kolisi adalah:

1) Linear Search

Dengan cara mencari tempat kosong terdekat dari alamat record yang mengalami kolisi.

2) Rehashing

Tersedia lebih dari satu fungsi Hash. Jika terjadi Kolisi, per hitungan dengan memakai fungsi Hash yang lain sampai tidak terjadi Kolisi lagi.

3) Area Overflow

Record yang mengalami kolisi ditempatkan pada aren Overflow Untuk menghubungkan file data dengan area Overflow diguna kan pointer.

Kriteria pemilihan fungsi transformasi :

- 1) Membutuhkan ruang seminimal mungkin
- 2) Menghindari tabrakan (Kolisi)

Jadi pada Direct File memungkinkan untuk mengakses sebuah record secara langsung dengan menggunakan Key Attribute tanpa mem pertimbangkan posisi Key tersebut dalam urutannya di keseluruhan file.

6. Ringkasan

- 1) Pengaksesan paling cepat pada file adalah dengan mengetahui Alamat dari record yang akan diakses atau diambil.

- 2) Untuk mencapai pengalamatan secara langsung kunci dari record digunakan untuk melokasikan record dalam file. Direct Access diagram bisa dilihat digambar 13-1
- 3) Metode Direct File menggunakan perhitungan t untuk menyediakan alamat record dengan satu Kunci.
- 4) Pemrosesan file secara langsung, atau akses langsung, memungkinkan komputer secara langsung menempatkan record yang diinginkan dengan menggunakan satu kunci record.
- 5) Pemrosesan secara langsung memerlukan penyimpanan disk, di mana devise disknya disebut devise penyimpanan akses langsung (direct-access storage device/DASD) karena komputer bisa secara langsung menempatkan record pada disk.
- 6) Keuntungan tambahan dari akses langsung adalah kemampuan untuk membaca, mengubah, dan menghasilkan satu record pada tempat yang sama di disk, dimana hal ini disebut sebagai pemrosesan langsung di tempat.

C. Soal Latihan

1. Apa pengertian dari organisasi berkas langsung?
2. Apa itu Fungsi Hash pada organisasi berkas langsung?
3. Apa yang dimaksud kunci sebagai alat rekaman yang unik?

D. Referensi

Handayani, Dewi. 2001. Sistem Berkas. Yogyakarta. J&J Learning
Alan I, Tharp-john Willey & Son. 1988. File Organization and Processing.